SYSTEM INTEGRATION ASSIGNMENT

ToursApplication — Integration Architecture

Outbound Integration, API Security, and Inbound REST Processing

FINKI — Faculty of Computer Science and Engineering

Reference document: explains what was built and the reasoning behind each layer

What the assignment actually asked

The assignment has three independent parts, each addressing a different direction of communication between systems.

PART 1 — OUTBOUND INTEGRATION

```
Our System
     |
     v
Comments System
```

When a client calls `GET /api/consultation/{id}`, our system must also fetch comments from another system and merge them into the response. We are the caller, so this is **outbound**.

PART 2 — SECURITY

Other systems will call our APIs. We don't want everybody to access them, so we need **API Keys** and **Rate Limiting** to protect the system.

PART 3 — INBOUND INTEGRATION

```
Faculty A
     |
Faculty B
     |
Faculty C
     |
     v
Our System
```

Other faculties send attendance data to us. Data is coming in, so this is **inbound**.

Part 1 — Outbound Integration

PART 1

STEP 1 Create ConsultationApiSettings

Domain → Configuration → ConsultationApiSettings.cs

```
public class ConsultationApiSettings
{
    public string BaseAddress { get; set; }
    public string ApiKey { get; set; }
    public int TimeoutSeconds { get; set; }
    public int CacheExpirationMinutes { get; set; }
}
```

> **Why:** hardcoding a URL like the consultation API address everywhere means that if it ever changes, the entire project must be searched and edited. Reading it from configuration means only the config value changes — no code changes.

STEP 2 Add configuration to appsettings.json

Web → appsettings.json

```
"ConsultationApi": {
  "BaseAddress": "https://integriranisistemi.finki.ukim.mk",
  "TimeoutSeconds": 30,
  "CacheExpirationMinutes": 60
}
```

> **Why:** ASP.NET reads configuration from appsettings.json and binds it into typed objects: JSON → ConsultationApiSettings → used in code.

STEP 3 Register the settings

Program.cs

```
builder.Services.Configure<ConsultationApiSettings>(
    builder.Configuration.GetSection("ConsultationApi"));
```

> **Why:** without this line, ASP.NET has no idea that the "ConsultationApi" section maps to the ConsultationApiSettings class. This line creates that connection.

STEP 4 Create the DTO

Domain → Dto → ConsultationCommentDto.cs

```
public class ConsultationCommentDto
{
    public Guid Id { get; set; }
    public string Comment { get; set; }
    public DateTime CreatedAt { get; set; }
}
```

> **Why:** Controllers should never talk directly to database entities or raw external responses. DTOs are the application's own language: External API → Response Object → DTO → Service.

STEP 5 Create response classes

Domain → ConsultationApiResponse (ConsultationApiResponse, ConsultationCommentResponse)

> **Why:** when the external API returns JSON such as `{ "id": ..., "comments": [...] }`, ASP.NET needs a class describing where each field goes. This is called **deserialization**: JSON → Response Class.

STEP 6 Create IConsultationApiClient

Service → Interface

> **Why:** code is written against interfaces, not concrete classes. Controller → IConsultationApiClient → ConsultationApiClient means the real implementation can later be swapped for a fake/test version without touching any calling code.

STEP 7 Create ConsultationApiClient

Service → Implementation

Its job is narrow and singular — it does nothing besides call the external API:

```
Build URL
   ↓
Send HTTP Request
   ↓
Receive JSON
   ↓
Convert JSON
   ↓
Return Data
```

> **Why:** no caching, no database access, no business logic here — only the HTTP call.

STEP 8 Register HttpClient

Program.cs

> **Why:** every outgoing request needs the `X-Api-Key` header. Configuring the HttpClient once means it is attached automatically rather than added manually on every call.

STEP 9 Create ConsultationCommentService

```
Controller
   ↓
ConsultationCommentService
   ↓
ApiClient
```

> **Why:** a common mistake is calling the ApiClient directly from the Controller. Business logic — caching, validation, logging — belongs in a service layer, not inside the ApiClient.

STEP 10 Add caching

The assignment requires a refresh interval of one hour, implemented with `IMemoryCache`.

```
Request
   ↓
Check Cache ──found──► Return Cached Data
   |
  not found
   ↓
Call External API
   ↓
Store Result
   ↓
Return Result
```

> **Why:** external API calls are expensive, and the assignment explicitly asks to reduce the number of calls made.

Part 1 summary

```
ConsultationController
        ↓
ConsultationService
        ↓
ConsultationCommentService
        ↓
      (Cache)
        ↓
ConsultationApiClient
        ↓
External Comments System
```

Part 2 — Security

PART 2

If an endpoint such as `POST /api/external/attendance` were left unsecured, anyone could send data to it. API Keys and Rate Limiting close that gap.

STEP 11 Create ApiClient model

Domain → Models

Represents an external faculty/application/system permitted to call our API — e.g. Faculty of Medicine, Faculty of Economics, Faculty of Law — each issued its own key.

STEP 12 Add DbSet

ApplicationDbContext → DbSet<ApiClient>

> **Why:** Entity Framework only tracks entities registered in the DbContext. Without a DbSet, there is no table and no queries.

STEP 13 Create migration

> **Why:** models only exist in code until a migration creates the corresponding `ApiClients` table inside SQLite.

STEP 14 Create ApiKeyMiddleware

```
Request
   ↓
Middleware
   ↓
Controller
```

The middleware checks whether the request contains `X-Api-Key`. If missing → **401 Unauthorized**. If present, it looks the key up in the database; found → continue, not found → **401 Unauthorized**.

STEP 15 Rate limiting

> **Why:** without a limit, a burst of thousands of requests per second could bring the server down. The rate limiter caps requests (e.g. 5–10 per minute, depending on configuration).

Part 2 summary

```
Request
   ↓
ApiKeyMiddleware
   ↓
Validate API Key
   ↓
Rate Limiter
   ↓
Controller
```

Part 3 — Inbound REST

PART 3

The direction reverses: in Part 1 we call them; in Part 3, they call us.

STEP 16 Create InboundAttendanceRequest

Represents the incoming JSON payload:

```
{
  "userId": "1",
  "consultationId": "2",
  "attendedAt": "...",
  "notes": "Present"
}
```

> **Why:** ASP.NET needs a defined shape to convert incoming JSON into a C# object.

STEP 17 Create ExternalAttendanceController

- `POST /api/external/attendance`
- `GET /api/external/attendance/{id}/status`

> **Why:** these two routes are explicitly required by the assignment.

STEP 18 Create InboundAttendanceEntry

> **Why this matters:** processing is not immediate. An Attendance record is not created instantly — the request is received, stored, and processed later. This is **asynchronous processing**.

```
Receive Request
      ↓
Store Request
      ↓
Process Later
```

RawPayload preserves the exact JSON originally received, so that if processing fails later, the original input is still recoverable. **Status** tracks the entry's progress through the pipeline: Pending → Processing → Completed / Failed.

STEP 19 Save the request

On arrival: create an InboundAttendanceEntry with status **Pending**, persist it, and respond with **202 Accepted** — the request was received, but not yet processed.

STEP 20 Create the processing service

Finds pending requests and converts them into Attendance records.

STEP 21 Quartz job

> **What Quartz is:** a scheduler — effectively an alarm clock for the application — that wakes up every 30 seconds.

STEP 22 Process 5 entries per run

Each run, Quartz requests 5 Pending entries and advances each one through the pipeline:

```
Pending → Processing → Create Attendance → Completed On failure: Pending → Processing → Exception → Failed
(ErrorMessage saved)
```

Part 3 summary

```
External Faculty
        ↓
   POST Attendance
        ↓
ExternalAttendanceController
        ↓
InboundAttendanceEntry
        ↓
  Database (Pending)

  Every 30 seconds
        ↓
     Quartz Job
        ↓
InboundProcessingService
        ↓
  Attendance Created
        ↓
     Completed
```

The Whole Story in One Picture

Part 1 — Outbound

```
ConsultationController
        ↓
ConsultationService
        ↓
ConsultationCommentService
        ↓
      (Cache)
        ↓
ConsultationApiClient
        ↓
External Comments System
```

Part 2 — Security

```
Request
   ↓
ApiKeyMiddleware
   ↓
Validate API Key
   ↓
Rate Limiter
   ↓
Controller
```

Part 3 — Inbound

```
External Faculty
        ↓
   POST Attendance
        ↓
ExternalAttendanceController
        ↓
InboundAttendanceEntry
        ↓
  Database (Pending)

  Every 30 seconds
        ↓
     Quartz Job
        ↓
InboundProcessingService
        ↓
  Attendance Created
        ↓
     Completed
```

This explanation is built to hold up in an oral exam: it covers not just what was created, but why each layer exists and what responsibility it carries within the architecture.